베이지안 기계학습

Gaussian processes

조성일

인하대학교
통계학과

Table of contents

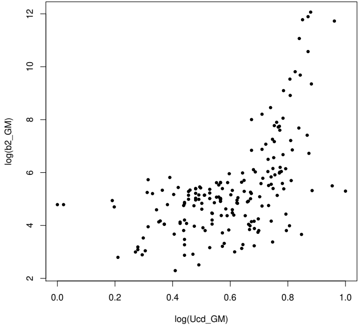
Introduction

Motivation I

- We observe $\{(y_i, x_i),\ i = 1, \ldots, n\}$

$$y_i = f(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathrm{N}(0, \sigma^2)$$

- To estimate $f(x)$, we need a prior for f.



Motivation II

- A prior for f involves specifying a distribution for every possible value of $f(x)$, with $x \in \mathcal{S} \subset \mathbb{R}^p \Rightarrow$ A stochastic process on an uncountable space.

Motivation II

- A prior for f involves specifying a distribution for every possible value of $f(x)$, with $x \in \mathcal{S} \subset \mathbb{R}^p \Rightarrow$ A stochastic process on an uncountable space.
- Stochastic processes on uncountable spaces through joint distributions on countable collections $\Rightarrow$ Kolmogorov consistency conditions:
  - Symmetry
  - Marginalization

Gaussian Processes (GPs)

GPs: Definition I

- Gaussian processes (GPs) provide a natural method to specify distributions on the space of functions (O'hagan (1978), Wahba (1978), and Rasmussen and Williams (2006)).

- $\{f(x) : x \in \mathcal{X}\} \sim \mathrm{GP}(\mu(x), \nu(x, x'))$ if and only if for any n and $x_1, \ldots, x_n \in \mathcal{X}$

$$\begin{pmatrix} f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix} \sim \mathrm{N}\left(\begin{pmatrix} \mu(x_1) \\ \vdots \\ \mu(x_n) \end{pmatrix}, \begin{pmatrix} \nu(x_1, x_1) & \cdots & \nu(x_1, x_n) \\ \vdots & \ddots & \vdots \\ \nu(x_n, x_1) & \cdots & \nu(x_n, x_n) \end{pmatrix} \right)$$

GPs: Definition II

- (Interpretability) μ represents the prior "expected form" for f, and ν controls how close the realizations are to μ.

- (Conjugacy) Assuming you know μ and ν,

$$f(x) \mid y_1, \ldots, y_n \sim \mathrm{GP}(\mu_n(x), \nu_n(x, x')).$$

GPs: Covariance kernel I

- A function $\nu : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is called a kernel, if
  - (symmetry)

$$\nu(s, t) = \nu(t, s), \text{ for all } t, s \in \mathcal{X}.$$

  - (nonnegativity)

$$\nu(s, t) \geq 0, \text{ for all } t, s \in \mathcal{X}.$$

GPs: Covariance kernel I

- A function $\nu : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is called a kernel, if
  - (symmetry)
    $$\nu(s,t) = \nu(t,s), \text{ for all } t, s \in \mathcal{X}.$$
  - (nonnegativity)
    $$\nu(s,t) \geq 0, \text{ for all } t, s \in \mathcal{X}.$$
- A kernel ν is called a Mercer kernel, if for all $x_1, \ldots, x_n \in \mathcal{X}$ and $n \in \mathbb{N}$, $G = (\nu(x_i, x_j))_{i,j=1}^n$ is positive definite.
- G is called the Gram matrix.

GPs: Covariance kernel II

- For real-valued inputs, $\mathcal{X} = \mathbb{R}^n$, it is common to use stationary kernels, which are functions of the form $\nu(s,t) = \nu(r)$, where $r = s - t$.

- Squared exponential (radial basis function, RBF) kernel

$$\nu(r;l) = \exp\left(-\frac{r^2}{2l^2}\right),$$

where $r = \|s - t'\|$ and l corresponds to the length-scale of the kernel, i.e., the distance over which we expect differences to matter.

GPs: Covariance kernel III

- Automatic relevance determination (ARD) kernel

$$\nu(r; l, \tau^2) = \tau^2 \exp\left(-\frac{1}{2}\sum_{i=1}^{n}\frac{r_i^2}{l_i^2}\right),$$

where τ^2 is interpreted as the overall variance.

GPs: Covariance kernel IV

- Matérn kernels

$$\nu(r;\delta,l) = \frac{2^{1-\delta}}{\Gamma(\delta)}\left(\frac{\sqrt{2\delta}r}{l}\right)^{\delta} K_{\delta}\left(\frac{\sqrt{2\delta}r}{l}\right),$$

where K_δ is a modified Bessel function.

GPs: Covariance kernel V

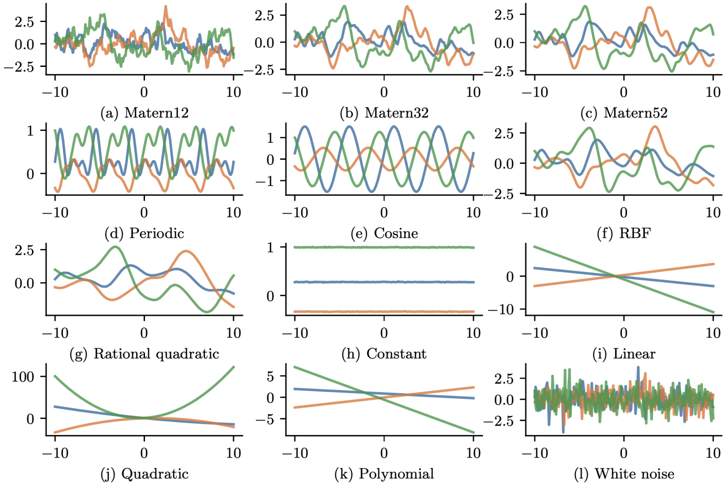


Figure: Murphy (2023). Figure 18.4 in Chapter 18.

GPs: Basis expansion I

- The Karhunen-Loéve representation is an infinite series expansion:

$$f(x) = \sum_{j=0}^{\infty} \theta_j \varphi_j(x)$$

where $\{\varphi_j\}$ forms an orthonormal basis on $[0, 1]$.

GPs: Basis expansion II

- The spectral coefficients are defined by:

$$\theta_j = \int_0^1 f(x)\bar{\varphi}_j(x)dx \text{ for } j \geq 0$$

where $\bar{\varphi}_j$ is the complex conjugate of φ_j.

GPs: Basis expansion II

- The spectral coefficients are defined by:

$$\theta_j = \int_0^1 f(x)\bar{\varphi}_j(x)dx \text{ for } j \geq 0$$

where $\bar{\varphi}_j$ is the complex conjugate of φ_j.

- Their variances ν_j^2 are derived from the covariance function:

$$\nu_j^2 = \int_0^1 \int_0^1 \nu(s,t)\varphi_j(s)\bar{\varphi}_j(t)ds\,dt, \;\; \sum_{j=0}^{\infty} \nu_j^2 < \infty.$$

GPs: Basis expansion III

- Conversely, the covariance function is a linear function of products of the basis functions with weights ν_j^2:

$$\nu(s,t) = \sum_{j=0}^{\infty} \nu_j^2 \varphi_j(s) \bar{\varphi}_j(t)$$

provided $\sum_{j=0}^{\infty} \nu_j^2 < \infty$, which is also known as Mercer's theorem (Mercer, 1909).

GPs: Cosine basis function (Lenk, 1999)

Cosine basis function is an orthonormal system defined on $[0, 1]$:

$$\varphi_0(x) = 1 \text{ and } \varphi_j(x) = \sqrt{2}\cos(\pi j x) \text{ for } j \geq 1 \text{ and } 0 \leq x \leq 1.$$

- By excluding sines, f does not have to be periodic on $[0, 1]$.
- All piecewise continuous function can be expressed as in infinite series of $\{\varphi_j\}$.
- If the support of x is $S = [a, b]$, then the orthonormal basis can be defined as:

$$\varphi_0(x) = \sqrt{\frac{1}{b-a}} \text{ and } \varphi_j(x) = \sqrt{\frac{2}{b-a}}\cos\left[\pi j\left(\frac{x-a}{b-a}\right)\right] \text{ for } j \geq 1.$$

Sparse Gaussian Process (SGP)

SGP: Introduction I

- Consider a nonparametric regression model with a GP prior (GP regression)

$$
\begin{aligned}
y_i \mid f(x_i) &\sim \mathcal{N}(f(x_i), \sigma^2) \\
f = (f(x_1), \ldots, f(x_n))^\top &\sim \mathcal{GP}(0, \nu).
\end{aligned}
$$

SGP: Introduction I

- Consider a nonparametric regression model with a GP prior (GP regression)

$$\begin{aligned} y_i \mid f(x_i) &\sim \mathcal{N}(f(x_i), \sigma^2) \\ f = (f(x_1), \dots, f(x_n))^\top &\sim \mathcal{GP}(0, \nu). \end{aligned}$$

- The joint density of the observed data and new data is given by

$$\begin{pmatrix} y \\ f_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \nu + \sigma^2 I & \nu_* \\ \nu_*^\top & \nu_{**} \end{pmatrix} \right)$$

SGP: Introduction II

- The posterior predictive density at new data is

$$
\begin{aligned}
p(f_* \mid \mathcal{D}, x_*) &= \mathcal{N}(f_* \mid \mu_{*|x}, \Sigma_{*|x}) \\
\mu_{*|x} &= \nu_*^\top (\nu + \sigma^2 I)^{-1} y \\
\Sigma_{*|x} &= \nu_{**} - \nu_*^\top (\nu + \sigma^2 I)^{-1} \nu_* .
\end{aligned}
$$

SGP: Introduction III

- The way to perform GP inference and training is to compute the inverse of the $n \times n$ Gram matrix.
- Unfortunately, this takes $O(n^3)$.
- We need methods to scale up GPs to handle large n.

SGP: Inducing point method I

- Sparse GP (SGP) is an approximation method based on inducing points (also called pseudoinputs).
- Let $z = (z_1, \ldots, z_M)^\top$ be $M(< n)$ additional inputs with unknown function value f_z (often denoted by u).

SGP: Inducing point method I

- Sparse GP (SGP) is an approximation method based on inducing points (also called pseudoinputs).
- Let $z = (z_1, \ldots, z_M)^\top$ be $M(< n)$ additional inputs with unknown function value f_z (often denoted by u).
- The exact joint prior has the form

$$
\begin{aligned}
p(f, f_*) &= \int p(f_*, f, f_z) df_z = \int p(f_*, f \mid f_z) p(f_z) df_z \\
&= \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \nu & \nu_* \\ \nu_*^\top & \nu_{**} \end{pmatrix} \right).
\end{aligned}
$$

SGP: Inducing point method II

- SGP predicts f_* just using f_z instead of f, i.e., assumes $f_* \perp f \mid f_z$.
- An approximation of the prior is given by

$$p(f_*, f, f_z) = p(f_* \mid f, f_z)p(f \mid f_z)p(f_z) \approx p(f_* \mid f_z)p(f \mid f_z)p(f_z).$$

SGP: Inducing point method II

- SGP predicts f_* just using f_z instead of f, i.e., assumes $f_* \perp f \mid f_z$.
- An approximation of the prior is given by

$$p(f_*, f, f_z) = p(f_* \mid f, f_z)p(f \mid f_z)p(f_z) \approx p(f_* \mid f_z)p(f \mid f_z)p(f_z).$$

- The train and test conditionals are

$$\begin{aligned} p(f \mid f_z) &= \mathcal{N}(f \mid \nu_{x,z}\nu_{z,z}^{-1}f_z, \nu_{x,x} - \nu_{x,z}\nu_{z,z}^{-1}\nu_{z,x}) \\ p(f_* \mid f_z) &= \mathcal{N}(f_* \mid \nu_{*,z}\nu_{z,z}^{-1}f_z, \nu_{*,*} - \nu_{*,z}\nu_{z,z}^{-1}\nu_{z,*}) \end{aligned}$$

SGP: Inducing point method III

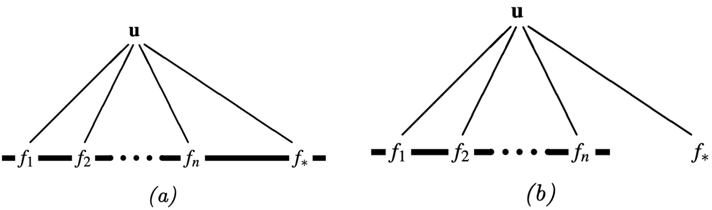


Figure: Figure 18.12 in Chapter 18. of Murphy (2023)

SGP: Inducing point method IV

- The inducing points or pseudoinputs can be treated like kernel hyperparameters.
- The log marginal likelihood is given by

$$
\begin{aligned}
\log m(y \mid x, z) &= \log \int \int p(y \mid f) p(f \mid x, f_z) p(f_z \mid z) df_z df \\
&= \log \int p(y \mid f) p(f \mid x, z) df \\
&= -\frac{1}{2} \log |Q + \sigma^2 I| - \frac{1}{2} y^\top (Q + \sigma^2 I)^{-1} y - \frac{n}{2} \log(2\pi),
\end{aligned}
$$

where $Q = \nu_{x,z} \nu_{z,z}^{-1} \nu_{z,x}$.

SGP: Inducing point method V

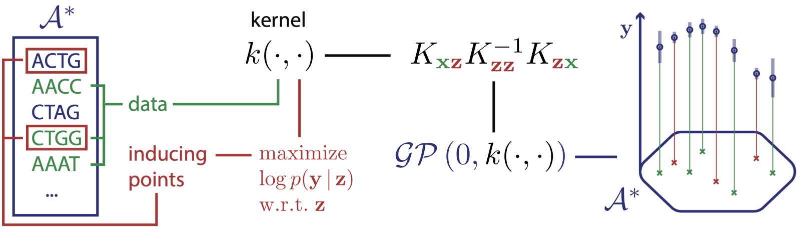


Figure: Figure 18.13 in Chapter 18. of Murphy (2023)

Example

Regression

- Regression model

$$y_i = f(x_i) + \epsilon_i, \;\; i = 1, \ldots, n,$$

where ϵ_i are errors following normal distribution with mean zero and variance σ^2. That is,

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

GP prior

Regression: GP prior I

- GP prior for f is
$$f = (f(x_1), \ldots, f(x_n))^\top \sim \mathcal{N}(0, \nu),$$
where ν is a squared exponential covariance kernel given as
$$\nu(x, x') = \tau^2 \exp\left(-\frac{1}{2}|x - x'|^2 / l^2\right).$$
- Priors for hyperparameters σ^2, τ^2, and l are set with log-normal distributions.

Regression: GP prior II

- GP regression - prior & model

```
pyro.clear_param_store()
kernel = gp.kernels.RBF(
   input_dim=1, variance=torch.tensor(5.0),
   lengthscale=torch.tensor(10.0)
)
gpr = gp.models.GPRegression(X, y, kernel,
                             noise=torch.tensor(1.0))

# note that our priors have support on the positive reals
gpr.kernel.lengthscale = pyro.nn.PyroSample(dist.LogNormal(0.0, 1.0))
gpr.kernel.variance = pyro.nn.PyroSample(dist.LogNormal(0.0, 1.0))
gpr.noise = pyro.nn.PyroSample(dist.LogNormal(0.0, 1.0))
```

Regression: GP prior III

- GP regression - fitting

```
from pyro.infer import SVI, Trace_ELBO
from pyro.optim import Adam

# the way we setup inference is similar to above
optim = Adam({"lr": 0.005})
svi = SVI(gpr.model, gpr.guide, optim, loss=Trace_ELBO())
losses = []
variances = []
lengthscales = []
noises = []
num_steps = 2000 if not smoke_test else 2
for i in range(num_steps):
  variances.append(gpr.kernel.variance.item())
  noises.append(gpr.noise.item())
  lengthscales.append(gpr.kernel.lengthscale.item())
  losses.append(svi.step())
```

SGP prior

Regression: SGP prior I

- SGP prior for f is

$$
\begin{aligned}
f \mid f_z &\sim \mathcal{N}(\nu_{x,z}\nu_{z,z}^{-1} f_z, \tilde{\nu}), \\
f_z &\sim \mathcal{N}(0, \nu_{z,z}),
\end{aligned}
$$

where $\tilde{\nu} = \nu_{x,x} - \nu_{x,z}\nu_{z,z}^{-1}\nu_{z,x}$ and $f_z = (f(z_1), \ldots, f(z_M))^\top$.

Regression: SGP prior II

- SGP regression - prior & model

```
# initialize the inducing inputs
Xu = torch.arange(20.0) / 4.0

pyro.clear_param_store()
kernel = gp.kernels.RBF(input_dim=1, variance=torch.tensor(5.0),
   lengthscale=torch.tensor(10.0)
)
sgpr = gp.models.SparseGPRegression(X, y, kernel, Xu=Xu, jitter=1.0e-5)

# note that our priors have support on the positive reals
sgpr.kernel.lengthscale = pyro.nn.PyroSample(dist.LogNormal(0.0, 1.0))
sgpr.kernel.variance = pyro.nn.PyroSample(dist.LogNormal(0.0, 1.0))
sgpr.noise = pyro.nn.PyroSample(dist.LogNormal(0.0, 1.0))
```

Regression: SGP prior III

- GP regression - fitting

```
svi = SVI(sgpr.model, sgpr.guide, optim, loss=Trace_ELBO())
losses = []
locations = []
variances = []
lengthscales = []
noises = []
num_steps = 2000 if not smoke_test else 2
for i in range(num_steps):
    locations.append(sgpr.Xu.data.numpy().copy())
    variances.append(sgpr.kernel.variance.item())
    noises.append(sgpr.noise.item())
    lengthscales.append(sgpr.kernel.lengthscale.item())
    losses.append(svi.step())
```

Remark and next week

Remark

- Gaussian processes (GPs) provide a natural method to specify distributions on the space of functions.
- The way to perform GP inference and training is to compute the inverse of the $n \times n$ Gram matrix, which takes $O(n^3)$.
- Sparse GP (SGP) is an approximation method, which is based on inducing points (also called pseudoinputs), to scale up GPs to handle large n.

Next week

- Variational AutoEncoder

References

- Pyro tutorials
- Murphy (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Murphy (2023). *Probabilistic Machine Learning: Advanced Topics*. MIT Press.